

HELPYARD.STORE

Professional Developer / Local Agent Implementation Plan

Source basis: Helpyard.store — Professional E-commerce Website & Mobile-App Development Roadmap, 7 pages, prepared 04 October 2026.

Purpose: This document converts the supplied roadmap into an execution instruction for a professional local developer or coding agent. The developer must build, test, document and hand over the system in controlled phases—not jump directly into UI coding or a mobile app.

1. Non-Negotiable Product Direction

- Build ONE secure commerce platform with five commercial sections: Courses, Ready Websites, Official Software, Books, and Cyber-Security Devices.
- Do NOT build five disconnected applications. Shared users, cart, checkout, payments, orders, coupons, reviews, notifications and admin must remain centralized.
- Use a common products model with product_type values such as physical, digital, software, course and book; add type-specific tables only where business rules differ.
- Web platform + REST API comes first. PWA is next; native Flutter/React Native mobile app comes only after the API and web platform are stable.
- Private paid assets—software files, ebooks and course media—must not be directly exposed through public URLs; downloads must be authorization-controlled.
- Never trust browser-supplied price, quantity, discount or payment-success values. The PHP backend independently validates and calculates authoritative values.

2. Approved First-Version Technology Stack

Layer	Required direction
Frontend	HTML5, CSS3, Bootstrap 5.3, JavaScript ES6+, Fetch/AJAX
Backend	PHP 8.3+ using MVC-style architecture and service layer
Database	MySQL 8 or MariaDB
API	REST/JSON under /api/v1
Payments	Bangladesh-friendly gateway(s), with international gateway optional
Storage	Local object/file storage initially; S3-compatible storage later
Server	Apache or Nginx + PHP-FPM + SSL + MySQL
Mobile	PWA first; Flutter or React Native later
Dependencies	Composer and documented package versions

3. Developer Working Rules

- Before coding: confirm requirements, product types, pricing rules, delivery rules, refund rules, roles, shipping zones and payment methods.
- Create wireframes and the design system before implementing the full UI.
- Use Git from the first day. Keep main production-ready; use develop for integration; feature/* for individual features; fix/* for bugs/security fixes.
- Use separate development, staging and production environments. Never commit production secrets, payment secrets or database credentials.
- Keep business authority on the server. JavaScript is for user experience, not security or final business decisions.

- Use reusable components and services instead of duplicating logic across the five storefront sections.
- Every phase must end with a working build, tests, documentation and a clear acceptance decision before the next dependent phase begins.

4. Required Architecture

Target flow: Browser/PWA → Frontend → PHP Application → Service Layer → MySQL + File Storage → Payment / Email / Shipping integrations.

Module	Developer responsibility
Authentication	Registration, login, verification, password reset, sessions, roles
Catalog	Categories, products, variants, images, specifications, pricing, stock, SEO
Product Types	Physical, digital, software, course, book rules
Search	Search, filtering, category/product navigation
Cart	Persistent cart, quantity and stock validation
Checkout	Server-side totals, coupons, shipping, order creation and state machine
Payment	Gateway integration, transaction records, verified callbacks/webhooks, idempotency
Orders	Order lifecycle, history, fulfillment and refunds/cancellation states
Digital Delivery	Entitlements and protected downloads
Physical Fulfillment	Inventory, packing, shipping and delivery tracking
Courses	Enrollment, sections, lessons, progress, quizzes and certificates
Licensing	Software license generation/assignment
Reviews/Coupons	Centralized review and promotion rules
Notifications	Customer/admin notifications and relevant email events
Admin	Products, orders, customers, payments, inventory, content, reports, audit logs
API	Stable REST/JSON contract for future mobile client
Audit/Operations	Logs, audit events, backups, error handling and monitoring

5. Database Implementation Order

Create migrations first; do not hand-edit production tables. Core tables required by the roadmap:

- users, roles, permissions, user_addresses
- categories, products, product_variants, product_images, product_files
- licenses, inventory
- carts, cart_items, orders, order_items
- payments, payment_transactions, shipments
- coupons, coupon_usages, reviews, wishlists, notifications, support_tickets, audit_logs

Course-specific tables:

- courses, lessons, course_sections, enrollments, lesson_progress, quizzes, quiz_questions, quiz_attempts, certificates

Order lifecycle must explicitly support: pending → payment_pending → paid → processing → shipped/delivered OR digital_ready → completed, with cancelled/refunded and payment-failed states handled explicitly.

6. Required Project Structure

- /helpyard-store/public/ (index.php, public assets)
- /app/Controllers/ /app/Models/ /app/Services/ /app/Repositories/
- /app/Middleware/ /app/Validation/ /app/Views/
- /config/ /routes/web.php /routes/api.php
- /database/migrations/ /database/seeder/
- /storage/logs/ /storage/private/ /tests/
- /vendor/ .env composer.json README.md

Private files must be stored outside public paths where possible and served through an authorized backend download endpoint.

7. Phase-by-Phase Execution Roadmap

Phase 0 — Requirements & Wireframes

Work: Freeze five sections, product types, prices, delivery rules, refund rules, roles, shipping zones and payment methods. Produce sitemap, user flows and wireframes.

Definition of done: Approved requirements document; wireframes; acceptance rules; prioritized backlog.

Phase 1 — Engineering Foundation

Work: Initialize Git, PHP/MySQL environment, Composer, .env configuration, MVC routing, migrations, error handling, logging and coding standards.

Definition of done: Running local application; database migrations; documented setup; first Git history.

Phase 2 — UI/UX System

Work: Build responsive header, mega-menu, homepage, category cards, product cards, buttons, forms, modal/alerts, footer and mobile navigation as reusable components.

Definition of done: Responsive design system and reusable components at required breakpoints.

Phase 3 — Catalog & Product Engine

Work: Implement five catalog sections, universal product detail, categories, variants, images, specifications, pricing, stock, SEO fields, demos and downloadable-file metadata.

Definition of done: All five catalogs operate from one product engine.

Phase 4 — Accounts & Customer Dashboard

Work: Registration, email verification, login/logout, password reset, profiles, addresses, sessions, roles, orders, downloads, courses, licenses, wishlist and notifications.

Definition of done: Customer can securely authenticate and view only their own resources.

Phase 5 — Cart & Checkout

Work: Persistent cart, quantity validation, server-side price calculation, coupons, shipping calculation and order creation.

Definition of done: Checkout creates correct orders and rejects tampered client values.

Phase 6 — Payments

Work: Integrate selected gateways in sandbox first. Store transaction records; verify callbacks/webhooks server-side; implement idempotency and failure handling.

Definition of done: Only verified gateway results can mark an order paid.

Phase 7 — Fulfillment

Work: Physical: inventory → packing → shipping → delivery. Digital: entitlement → protected download. Software: license assignment. Courses: enrollment → lesson access.

Definition of done: Each product type follows its correct fulfillment path.

Phase 8 — Admin Panel

Work: Dashboard, product CRUD, categories, orders, payment status, inventory, customers, coupons, courses/content, downloads, reviews, support, reports and audit logs.

Definition of done: Admin can operate the platform without direct database edits.

Phase 9 — Search & SEO

Work: Clean URLs, metadata, canonical URLs, sitemap, robots.txt, Open Graph, structured data where appropriate and internal linking.

Definition of done: Search-engine-accessible category/product architecture.

Phase 10 — Security Hardening

Work: HTTPS, secure cookies, CSRF, prepared statements, output encoding, authorization, rate limiting, secure uploads, secret management, audit logs and backups.

Definition of done: Security test suite passes with no critical/high unresolved issues.

Phase 11 — QA & Acceptance

Work: Unit, integration, checkout, mobile browser, accessibility, payment sandbox, failure/retry, load and backup-restore tests.

Definition of done: Release candidate passes documented test matrix.

Phase 12 — Production Deployment

Work: Server, DNS, SSL, database migration, environment variables, permissions, cron jobs, monitoring, backups, error reporting and rollback.

Definition of done: Production smoke test passes; rollback procedure documented.

Phase 13 — PWA

Work: Installable web app, manifest, service worker, offline shell and push notifications where appropriate.

Definition of done: Stable PWA built on the same web/API platform.

Phase 14 — Native Mobile App

Work: Only after API stability. Build Flutter/React Native client using same auth, catalog, cart, orders, payments and notifications APIs.

Definition of done: Mobile client uses the existing backend; no duplicated business logic.

8. Required Route Map

Method / Route	Purpose
GET /	Homepage
GET /courses	Course catalog
GET /courses/{slug}	Course detail
GET /websites	Ready website catalog
GET /software	Software catalog
GET /books	Book catalog
GET /devices	Cyber-device catalog
GET /product/{slug}	Universal product detail
GET /cart	Cart
GET /checkout	Checkout
POST /checkout/order	Create order
POST /payment/callback	Verify gateway result
GET /account/orders	Customer orders
GET /account/downloads	Protected digital downloads
GET /admin	Admin dashboard
GET /api/v1/products	Products API
GET /api/v1/categories	Categories API
POST /api/v1/auth/login	API authentication
POST /api/v1/cart	API cart
POST /api/v1/orders	API order
GET /api/v1/orders/{id}	API order detail

9. Security Acceptance Rules

- Prepared statements or safe ORM query binding everywhere.
- CSRF protection on applicable state-changing web requests.
- Context-appropriate output encoding to prevent XSS.
- Strict authorization: customers cannot access admin functions or another customer's order/download.
- Secure session cookies, password hashing, session rotation and login throttling.
- Email verification; optional MFA for sensitive accounts.
- HTTPS everywhere.
- Upload MIME/type/size validation and randomized filenames; avoid public storage for private assets.
- Rate limiting for authentication, sensitive APIs and abuse-prone workflows.
- Centralized audit logging for important administrative/security events.
- Server recalculates price, quantity, stock and coupon eligibility.
- Payment callbacks/webhooks are verified server-side; browser success URLs never mark orders paid.
- Idempotency/concurrency protections prevent duplicate paid orders and negative inventory.
- Backups are created and restore-tested before production launch.

10. Testing & Launch Gate

Area	Must prove
Responsive	320, 375, 768, 1024 and desktop layouts work
Authentication	Registration, verification, login, logout, reset and session rotation
Authorization	Cross-user and customer/admin access isolation
Cart	Invalid quantities, stock changes, price changes and expired coupons rejected server-side
Checkout	Duplicate submission and repeated callback do not duplicate paid orders
Digital	Unauthorized direct downloads blocked; links are

	permission-checked/expirable
Physical	Concurrent orders cannot drive stock negative
Uploads	MIME/type/size checks and safe storage
Security	CSRF, XSS, SQL injection, access-control, session, rate-limit and business-logic testing
Operations	Backup, restore, logs, monitoring, error pages and rollback procedure

11. Git / Agent Workflow

- main = production-ready code.
- develop = integration branch.
- feature/* = one feature at a time.
- fix/* = bug/security fixes.
- Use small, descriptive commits such as: feat: add course catalog; feat: add secure digital downloads; feat: integrate payment callback verification; fix: prevent duplicate order creation; security: enforce admin authorization; docs: update deployment guide.
- Every completed feature must include code, migration changes if required, validation, tests, documentation and a manual verification note.

12. Local Coding-Agent Instruction

Give the local agent the following operating rule:

“Act as a senior full-stack PHP e-commerce engineer. Implement Helpyard.store strictly as one shared commerce platform following this document. Before changing code, inspect the existing repository and identify architecture, database state and current features. Never overwrite working functionality blindly. Work one phase/feature at a time. For every change: explain the goal, inspect dependencies, implement the smallest maintainable change, run relevant tests, perform security validation, update documentation/migrations, and report files changed plus verification results. Server-side validation is authoritative. Never expose private paid assets publicly. Never mark payments paid from browser redirects. Never duplicate business logic for future mobile apps; expose reusable REST API behavior. Stop and request a decision only when a requirement is genuinely missing or conflicts with the source roadmap.”

13. Definition of Done for the First Production Release

- Polished responsive homepage and navigation.
- Courses, Ready Websites, Official Software, Books and Cyber-Security Devices are live as five sections of the same platform.
- Universal product detail and search/filter are operational.
- Customer accounts, cart and checkout are operational.
- At least one reliable payment integration is verified end-to-end in production-safe flow.
- Physical and digital fulfillment work according to product type.
- Customer order history and secure downloads work.
- Admin can manage products and orders.
- SEO basics, HTTPS, backups, logs and security testing are complete.
- Deployment, rollback and operational documentation are complete.

14. Explicitly Defer Until Core Platform Is Stable

- Advanced subscriptions.
- Vendor / multi-seller support.

- Advanced course LMS capabilities beyond the first required course flow.
- Advanced automated software licensing beyond the required first implementation.
- Recommendation engine.
- PWA push enhancements beyond the MVP.
- Advanced analytics dashboards.
- Native Flutter/React Native app.

15. Final Developer Handoff Package

- Source code repository with clean Git history.
- Database migrations and seeders.
- .env.example with secrets omitted.
- API route/contract documentation.
- Setup and deployment README.
- Admin and customer user guide.
- Test report and launch checklist.
- Security test findings and remediation status.
- Backup/restore and rollback instructions.
- List of third-party services, package versions and configuration requirements.

16. Source-Derived Architecture Summary

The supplied roadmap defines the target as one secure commerce platform with multiple product types, shared commerce infrastructure, a REST API, protected digital delivery and a staged web → PWA → native-app path. The implementation above preserves that direction while making the work actionable for a professional developer or local coding agent.

Source reference: Helpyard.store Professional Developer Roadmap, especially the architecture, database, phased roadmap, security, testing, Git workflow, deployment, build order and MVP sections.